

Unit : 2

Prof. Ashwani Mishra

Assistant Professor
AIML Department

CONTENTS

- Processes
- Process Scheduling
- Operation on Processes
- Interprocess Communication
- Threads
- CPU Scheduling
- Process Synchronization

Processes

A process is more than the program code, known as the text section. A process includes the current activity, represented by the value of the program counter, the contents of the processor's registers, the process stack, which contains temporary data, and a data section which contains global variables.

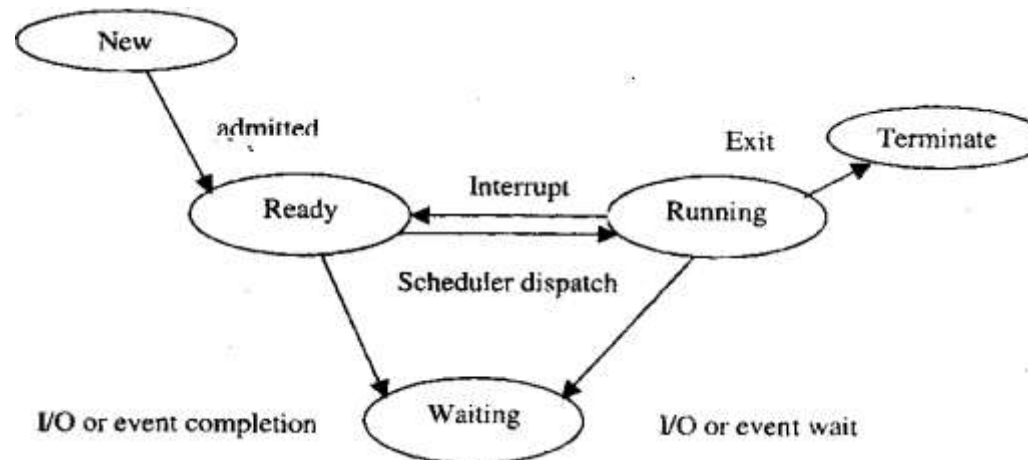
CPU-I/O Burst Cycle

- The following property of a process made the multiprocessing successful by utilising the CPU to the maximum extent. Every process executes in cycles of CPU execution (CPU Burst) and I./O wait (I/O Burst).
- Process execution starts with a CPU burst followed by an I/O burst through repeated cycles. Eventually, the last CPU cycle will terminate the process. It was observed that most of the processes will have many smaller CPU bursts.
- Based on the duration of the CPU bursts, a process can be either classified as a CPU-bound process if it has a few very long CPU bursts or it may be classified as I/O bound process if it has many very short CPU bursts.
- **Process State** - As a program executes, it changes state. The **state** of a process is defined in part by the current activity of that process.
 - a. New:** The process is being created.
 - b. Running:** Instructions are being executed.
 - c. Waiting:** The process is waiting for some event to occur.
 - d. Ready:** The process is waiting to be assigned to a processor.
 - f. Terminated:** The process has finished execution.

CONTINUED ...

Process Control Block

- Each process is represented in the Operating system by a process control block (PCB), also called a task control block.
- PCB Contains many pieces of information associated with a specific process, including these:
 1. **Process state:** The state may be new, ready, running, waiting, halted, and so on
 2. **Program counter:** Indicates the address of the next instruction to be executed for this process.
 3. **CPU registers:** The registers vary in number and type, depending on the computer architecture, and include accumulators, index registers, stack pointers, and general-purpose registers, that pulse any condition-code information.



PROCESS SCHEDULING

Scheduling Queues

- As processes enter the system, they are put into a job queue, which consists of all processes in the system. The processes that are residing in the main memory and are ready and waiting to execute are kept on a list called the ready queue, generally stored as a linked list.
- A ready-queue header contains pointers to the first and final PCBs in the list. Extend each PCB to include a pointer field that points to the next PCB in the ready queue.
- The list of processes waiting for a particular I/O device is called a device queue.

Schedulers

- A process migrates between the various scheduling queues throughout its lifeline. The OS must select, processes from these queues in some fashion. The selection process is carried out by the appropriate Scheduler.

Types of Scheduler-

- a. The long-term scheduler, or job scheduler, selects processes from the pool where the processes are spooled to a mass-storage device (typically a disk) and load them into memory for execution.
- b. The short-term scheduler, or CPU scheduler, selects from among the processes that are ready to execute and allocates the CPU to one of them.

PROCESS CREATION

- A process may create several new processes via a create-process system call, during the course of execution. The creating process is called a **parent** process, and the new processes are called the **children** of that process. Each of these new processes may, in turn, create other processes, forming a tree of processes.
- A process will need certain resources (CPU time, memory, files, and I/O devices) to accomplish its task.

When a process is created, it obtains, in addition to the various physical and logical resources, initialisation data (or input) that may be passed along from the parent process to the child process.

Process Termination

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit system call. Then, the process may return data (output) to its parent process.
- All the resources of the process, including physical and virtual memory, open files, and I/O buffers, are deallocated by the operating system.
- A parent needs to know the identities of its children. Thus, when one process creates a new process, the identity of the newly created process is passed to the parent.

Cooperating processes

The concurrent processes executing in the operating system may be either independent processes or cooperating processes.

- A process is independent if it cannot affect or be affected by the other processes executing in the system.
- A process is cooperating if it can affect or be affected by the other processes executing in the system.

INTERPROCESS COMMUNICATION

Message Passing System

- The function of a system is to allow processes to communicate with one another without the need to resort to shared data.
- Here, services operate outside of the kernel.

An IPC facility provides at least two operations: Send (message) and Receive (message)

Messages sent by a process can be of either fixed or variable size.

- In fixed-sized messages, the system-level implementation is straightforward. This makes the task of programming more difficult.
- In variable-sized messages requires a more complex system-level implementation, but the programming task becomes simpler.
- Several methods for logically implementing a link and the send / receive operations:
 1. Direct or indirect communication.
 2. Symmetric or asymmetric communication.
 3. Automatic or explicit buffering.
 4. Send by copy or send by references.
 5. Fixed-sized or variable-sized messages.

DIRECT COMMUNICATION

- Each process that wants to communicate must explicitly name the recipient or sender of the communication. Here, the send and receive primitives are defined as Send (P, message) - Send a message to process P.

Receive (Q, message) - Receive a message from process Q.

- A communication link in this scheme has the following properties-
 - a. A link is established automatically between every pair of processes that want to communicate.
 - b. A link is associated with exactly two processes.
- A variant of this scheme employs asymmetry in addressing. Only the sender names the recipient; the recipient is not required to name the sender. Here, the send and receive primitives are defined as:
 - a. Send (P, message) - Send a message to process P
 - b. Receive (id, message) - Receive a message from any process; the variable id is set to the name of the process with which communication has taken place.

INDIRECT COMMUNICATION

- The messages are sent to and received from **mailboxes**, or **ports**. A mailbox can be viewed as an object into which messages can be placed by processes and from which messages can be removed. Each mailbox has a unique identification.
- Two processes can communicate only if they share a mailbox. The send and receive primitives are defined as follows:
 - a. Send (A, message) - Send a message to mailbox A.
 - b. Receive (A, message) - Receive a message from mailbox A.

A communication link in this scheme has the following properties:

- a. A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- b. A link is associated with more than two processes.
- c. A number of different links may exist between each pair of communicating processes, with each link corresponding to one mailbox.

THREADS

- A thread, sometimes called a lightweight process (LWP), is a basic unit of CPU utilisation; it comprises a thread ID, a program counter, a register set, and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.
- A traditional (or heavyweight) process has a single thread of control. If the process has multiple threads of control, it can do more than one task at a time.

Benefits-

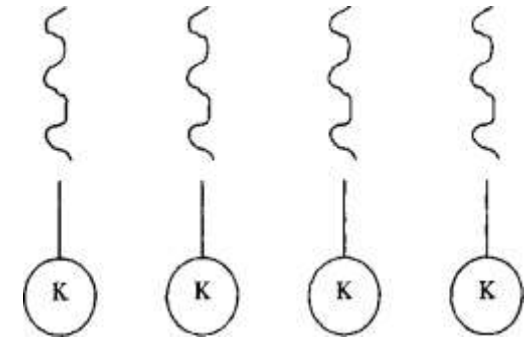
The benefits of multithreaded programming can be broken down into four major categories:

- a. Responsiveness:** Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.
- b. Resource sharing:** By default, threads share the memory and the resources of the process to which they belong.
- c. Economy:** It is more economical to create and context switch threads.
- d. Utilization of multiprocessor architectures:** Each thread runs in parallel on a different processor.

MULTITHREADING MODELS

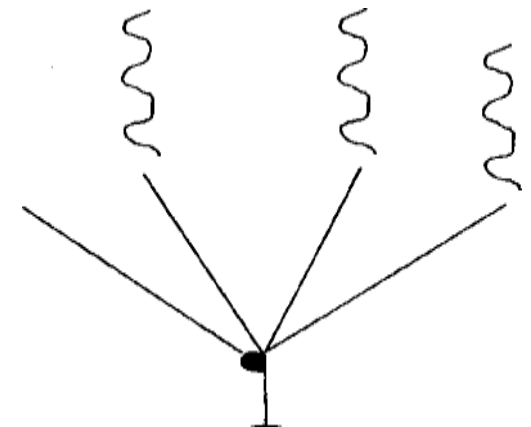
1. Many-to-One Model

- The many-to-one model maps many user-level threads to one kernel thread.
- Thread management is done in user space, so it is efficient, but the entire process will be blocked if a thread makes a blocking system call.
- Because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors. Green threads: A thread library available for Solaris 2 uses this model.



2. One-to-One Model

- It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call; it allows multiple threads to run in parallel on multiprocessors.
- The drawback is, creating a user thread requires creating the corresponding kernel thread. Windows NT, Windows 2000, and OS/2 implement the one-to-one model.
- It allows greater concurrency, but the developer has to be careful not to create many threads within an application.



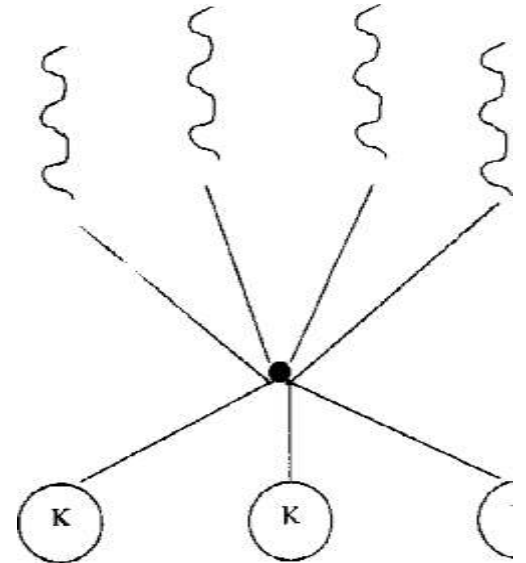
CONTINUED...

Many-to-Many Model

- The number of kernel threads may be specific to either a particular application or a particular machine.
- Developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.

Thread Pools –

- Unlimited threads could exhaust system resources, such as CPU time or memory. One solution to this issue is to use thread pools.
- A thread pool is to create a number of threads at process startup and place them into a pool, where they sit and wait for work. When a server receives a request, it awakens a thread from this pool, if one is available, passing it the request to service.
- Once the thread completes its service, it returns to the pool awaiting more work. If the pool contains no available thread, the server waits until one becomes free.



SOLARIS 2 THREADS

- Solaris 2 is a version of UNIX with support for threads at the kernel and user levels, SMP, and real-time scheduling.
- It implements the thread API in addition to supporting user-level threads with a library containing APIs for thread creation and management.
- It defines an intermediate level of threads as well. Between user- and kernel-level threads are lightweight processes (LWPs).
- Standard kernel-level threads execute all operations within the kernel. Solaris 2 implements the many-to-many model.
- User-level threads may be either bound or unbound.
 - a. A bonded user-level thread is permanently attached to an LWP.
 - b. An unbounded thread is not permanently attached to any LWP.
 - c. User-level threads are extremely efficient because no kernel support is required for thread creation or destruction, or for the thread library to context switch from one user-level thread to another.

LINUX THREADS

The Linux kernel introduced threads in version 2.2.

- It provides a fork system call with the traditional functionality of duplicating a process.
- It also provides the clone system call that is analogous to creating a thread.
- Clone behaves much like fork, except that instead of creating a copy of the calling process, it creates a separate process that shares the address space of the calling process.
- Through this sharing of the address space of the parent process, a cloned task behaves much like a separate thread.
- When a fork is invoked, a new process is created along with a copy of all the associated data structures of the parent process.

Java Threads-

Java is one of a small number of languages that provide support at the language level for the creation and management of threads.

Threads managed by the Java Virtual machine (JVM), not by a user-level library or kernel; it is difficult to classify Java threads as either user- or kernel-level.

CPU SCHEDULING

- In a single-processor system, only one process can run at a time; any others must wait until the CPU is free and can be rescheduled. The objective of multiprogramming is to have some process running at all times to maximise CPU utilisation. The idea is relatively simple.
- A process is executed until it must wait, typically for the completion of some I/O request. In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished. With multiprogramming, we try to use this time productively. Several processes are kept in memory at one time.
- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues. Every time one process has to wait, another process can take over the use of the CPU.

CPU-I/O Burst Cycle

- The success of CPU scheduling depends on an observed property of processes: Process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states. Process execution begins with a CPU burst. That is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst, and so on. Eventually, the final CPU burst ends with a system request to terminate execution.

CONTINUED..

Preemptive Scheduling

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of wait for the termination of one of the child processes)
2. When a process switches from the running state to the ready state (for example, when an interrupt occurs)
3. When a process switches from the waiting state to the ready state (for example, at completion of I/O).
4. When a process terminates.

- **Dispatcher**

Another component involved in the CPU-scheduling function is the dispatcher. The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

- **Scheduling Algorithms**

- CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU. There are many different CPU scheduling algorithms.
- By far, the simplest CPU-scheduling algorithm is the first-come, first-served (FCFS) scheduling algorithm. With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue. When a process enters the ready queue, its PCB is linked to the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue. The code for FCFS scheduling is simple to write and understand.

CONTINUED.....

- **Shortest-Job-First Scheduling**

A different approach to CPU scheduling is the shortest-job-first (SJF) scheduling algorithm. This algorithm associates with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling breaks the tie.

- **Priority Scheduling**

The SJF algorithm is a special case of the general priority scheduling algorithm.

A priority is associated with each process, and the CPU is allocated to the process with the highest priority. Equal-priority processes are scheduled in FCFS order. An SJF algorithm is simply a priority algorithm where the priority (p) is the inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.

- **Round-Robin Scheduling**

The round-robin (RR) scheduling algorithm is designed especially for timesharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum or time slice, is defined. A time quantum is generally from 10 to 100 milliseconds. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

Continued..

- **Multilevel Queue Scheduling**

Another class of scheduling algorithms has been created for situations in which processes are easily classified into different groups. For example, a common division is made between foreground (interactive) processes and background (batch) processes. These two types of processes have different response-time requirements, and so may have different scheduling needs. In addition, foreground processes may have priority (externally defined) over background processes.

- **Multilevel Feedback-Queue Scheduling**

Normally, when the multilevel queue scheduling algorithm is used, processes are permanently assigned to a queue when they enter the system. If there are separate queues for foreground and background processes, for example, processes do not move from one queue to the other, since processes do not change their foreground or background nature.

This setup has the advantage of low scheduling overhead, but it is inflexible.

- **Multiple-Processor Scheduling**

In the case of multi-processor Systems, the CPU scheduling is more complex, the system may contain some identical processes in their functionality, and some are heterogeneous.

Even if the system is homogeneous, accessing of an I/O device of a processor is a difficult thing if that processor is not working. If several processors are available, then there will be load sharing can occur, but careful protection of processes can be implanted; otherwise, two processes can take the same process and execute it.

PROCESS SYNCHRONIZATION

- A cooperating process is one that can affect or be affected by other processes executing in the system.
- Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through files or messages.
- The former case is achieved through the use of lightweight processes or threads.
- Concurrent access to shared data may result in data inconsistency.
- In this chapter, we discuss various mechanisms to ensure the orderly execution of cooperating processes that share a logical address space, so that data consistency is maintained.

The Critical-Section Problem

- Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$.
- Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, writing a file, and so on.
- The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section.
- That is, no two processes are executing in their critical sections at the same time.
- The critical-section problem is to design a protocol that the processes can use to cooperate.

CONTINUED...

Semaphores

- The various hardware-based solutions to the critical-section problem (using the TestAndSetC) and SwapO instructions) presented are complicated for application programmers to use. To overcome this difficulty, we can use a synchronisation tool called a semaphore.
- A semaphore S is an integer variable that, apart from initialisation, is accessed only through two standard atomic operations: wait () and signal (). The wait () operation was originally termed P (from the Dutch *proberen*, "to test"); signal () was originally called V (from *verhogen*, "to increment"). The definition of wait () is as follows:

```
Wait(S) : while S<=0;      //no-op  
          S-;
```

The definition of signal () is as follows:

```
signal(S) : S++;
```

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more -processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a signal() operation. When such a state is reached, these processes are said to be deadlocked.

CONTINUED...

- Suppose that P_0 executes `wait(S)` and then P_1 executes `wait(Q)`. When P_0 executes `wait(Q)`, it must wait until P_1 executes `signal(Q)`. Similarly, when P_1 executes `wait(S)`, it must wait until P_0 executes `signal(S)`. Since these `signal()` operations cannot be executed, P_0 and P_1 are deadlocked.

Classic Problems of Synchronization

- In this section, we present a number of synchronisation problems as examples of a large class of concurrency-control problems. These problems are used for testing nearly.
- Every newly proposed synchronisation scheme. In our solutions to the problems, we use semaphores for synchronisation.

The Bounded-Buffer Problem

- The bounded-buffer problem is commonly used to illustrate the power of synchronisation primitives. We present here a general structure of this scheme without committing ourselves to any particular implementation; we provide a related programming project in the exercises at the end of the chapter.
- We assume that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialised to the value 1.
- The empty and full semaphores count the number of empty and full buffers. The semaphore empty is initialised to the value n ; the semaphore full is initialised to the value 0.

CONTINUED...

Atomic Transaction

- A transaction is a program unit that must be executed atomically; that is, either all the operations associated with it are executed to completion, or none are performed. To ensure atomicity despite system failure, we can use a write-ahead log.
- All updates are recorded on the log, which is kept in stable storage. If a system crash occurs, the information in the log is used in restoring the state of the updated data items, which is accomplished by use of the undo and redo operations.
- To reduce the overhead in searching the log after a system failure has occurred, we can use a checkpoint scheme.
- **The Dining-Philosophers Problem**
- Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher.
- In the centre of the table is a bowl of rice, and the table is laid with five single chopsticks. When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbours).
- A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbour. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

THANK YOU